From Scenario to Code: Structured Prompting for LLM-Based Unit Test Generation

Aminata Diop*
Department of Mathematics and Computer Science
University of Quebec at Trois-Rivieres
Trois-Rivieres, Quebec, Canada
aminata.diop@uqtr.ca

Fadel Toure
Department of Mathematics and Computer Science
University of Quebec at Trois-Rivieres
Trois-Rivieres, Quebec, Canada
fadel.toure@uqtr.ca

Mourad Badri
Department of Mathematics and Computer Science
University of Quebec at Trois-Rivieres
Trois-Rivieres, Quebec, Canada
mourad.badri@uqtr.ca

Abstract

Automated unit test generation is crucial for improving software quality. Existing tools such as EvoSuite exhibits several limitations by generating tests that may lack readability and clarity. These tools may also struggle to cover specific code branches, particularly when complex code components are targetted. Furthermore, studies have shown that the generated unit tests may contain incorrect assertions or cause unexpected behaviors. To address these challenges, we propose a novel approach referred as Two-Step Zero-Shot Prompting (2SZSP), which leverages large language models (LLMs) to structure test generation by identifying relevant test scenarios and generating the corresponding unit test code. Our approach was evaluated on multiple object-oriented projects written in Java from the SBST 2020 dataset and compared to EvoSuite. The results show that, while our approach achieves lower code coverage, it generates more readable, better-contextualized tests with higher mutation score suggesting an increased ability to detect faults. These results pave the way for the integration of LLMs into automated testing pipelines, with the potential to improve the relevance of generated tests and their impact on software robustness.

CCS Concepts

• **Software and its engineering** → Software creation and management; Software verification and validation; Software defect analysis; Software testing and debugging; • **Computing methodologies** → Artificial intelligence; Natural language processing.

Keywords

Unit Test Generation, Unit Test Automation, LLM, Zero-Shot Prompting, EvoSuite, Athenatest, Software Quality

ACM Reference Format:
Aminata Diop, Fadel Toure, and Mourad Badri. 2025. From Scenario to Code: Structured Prompting for LLM-Based Unit Test Generation. In *2025 The 7th World Symposium on Software Engineering (WSSE 2025), October 24–26, 2025, Okayama, Japan.* ACM, New York, NY, USA, 8 pages. https://doi.org/10.1145/3779657.3779658

*Corresponding author.

This work is licensed under a Creative Commons Attribution 4.0 International License.
WSSE 2025, Okayama, Japan
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1577-8/2025/10
https://doi.org/10.1145/3779657.3779658

1 Introduction

Software testing is a critical phase in the software development lifecycle, ensuring the quality of software applications. Unit testing plays, in fact, a crucial role in validating individual components, preventing regressions, and improving software quality. However, writing unit tests is time-consuming, tedious, and requires significant efforts. To address these challenges, researchers have developed automated test generation tools, aiming at streamlining the process and reducing human intervention.

Among these tools, EvoSuite [1] and AthenaTest [2] have emerged as two major solutions. EvoSuite uses genetic algorithms to maximize code coverage, while AthenaTest, developed by Microsoft, leverages large language models (LLMs) such as BART [3] to generate human-like test cases. Despite their contributions, both tools face notable limitations.

EvoSuite, though effective in achieving high code coverage, struggles with complex object-oriented programs, often failing to reach certain execution paths. Studies report that up to 16.2% of defects may remain undetected due to path reachability issues [4]. Moreover, it frequently generates incorrect assertions or causes unexpected runtime exceptions, reducing its practical reliability [4]. A further concern is the low readability of the generated tests, which hinders adoption by developers [5].

Similarly, AthenaTest suffers from several shortcomings. While it introduces natural language understanding, it lacks advanced reasoning for assertions, leading to incorrect outputs in over 26% of test cases [6]. It also exhibits syntactic inconsistencies (e.g., invalid method signatures in ∼9% of outputs [6]) and shows limited contextual understanding, sometimes producing unrealistic or redundant tests [7]. Additionally, as it is trained on existing unit tests, AthenaTest struggles to generalize to unseen code patterns.

Beyond these tool-specific issues, automated test generation still faces broader challenges: the absence of oracles makes it difficult to validate test outcomes [4, 8]; complex scenarios involving object dependencies or state setups remain hard to cover; and tools often produce redundant or non-compilable tests, increasing noise [4].

Given these issues, a more effective approach to automated unit test generation is required—one that leverages recent advancements in artificial intelligence (AI) and machine learning (ML), particularly Large Language Models (LLMs), to overcome the limitations of existing tools. Recent advances in LLMs (e.g., GPT [9], BART [3], CodeBERT [10], Codex [11]) have opened new opportunities for test generation. Unlike static or search-based tools like EvoSuite [1] and Randoop [12], LLMs can generate context-aware tests that reflect the intended behavior of code. They also demonstrate better

assertion reasoning, having learned patterns from large corpora, and produce more readable and maintainable test code. Furthermore, modern LLMs can generalize to previously unseen methods, and reduce reliance on brute-force strategies.

However, as shown by AthenaTest, LLM-based methods still face key limitations. To address these gaps, we propose a novel approach referred as Two-Step Zero-Shot Prompting, which conducts test generation in 2 steps:

- Test scenario identification – The LLM first generates a set of possible test scenarios, ensuring broader test coverage.
- Test case synthesis – The LLM then generates unit test code specific to each identified scenario, improving the relevance and correctness of generated tests.

This structured approach leverages the flexibility of LLMs while aiming to improve assertion accuracy, test relevance, and compilability, ultimately leading to more effective automated test generation.

The rest of the paper is organized as follows. Section 2 reviews related work on automated unit test generation, with an emphasis on LLM-based techniques. Section 3 introduces our proposed Two-Step Zero-Shot Prompting approach. Section 4 describes the evaluation setup, including benchmark statistics and experimental methodology and compares our approach with existing tools. Section 5 discusses the results. Finally, Section 6 concludes the paper and outlines future research directions.

2 Related Work

Automatic unit test generation in object-oriented systems has been the subject of extensive research over the past decades, reflecting a wide range of approaches and technologies. This literature review aims to provide an overview of the major contributions in this field, highlighting the key methodologies, tools, and advancements that have shaped research in automated test generation.

2.1 Traditional Test Generation Approches

Automated test generation has been an active research field for several decades, leading to the development of various techniques aimed at improving test coverage and reducing developer effort. Among the most prominent approches, search-based test generation and random test generation have been widely explored.

Panichella et al. [13] introduced DynaMOSA (Dynamic Many-Objective Sorting Algorithm), an extension of the MOSA algorithm [14] designed to enhance the efficiency of unit test generation by dynamically focusing on uncovered objectives during the generation process. This approach reformulates the test generation task as a dynamic multi-objective optimization problem, where the objectives (e.g., branches to be covered) evolve based on the intermediate results obtained during test execution. The key innovation lies in the adaptive prioritization of uncovered targets, which enables faster and more focused maximization of code coverage. Search-based approaches often achieve better coverage than purely random testing, as they rely on optimization mechanisms guided by a cost function. However, these techniques require a well-defined and suitable cost function and can be more computationally expensive.

C. Pacheco and M.D. Ernst [12] introduced Randoop, a tool based on random test generation that dynamically improves by excluding irrelevant test cases. Their research demonstrated that Randoop can detect complex bugs with minimal human intervention. The approach uses a feedback mechanism to progressively enhance test generation: when a test sequence triggers an error, Randoop retains it and uses it as a basis for generating new, potentially more effective sequences. Pure random testing can quickly achieve basic code coverage but often struggles to cover complex scenarios that require very specific input values or precise method call sequences. To address this limitation, researchers and developers frequently incorporate feedback mechanisms or learning strategies to refine and guide the test generation process.

N. Tillmann and J. de Halleux [15] introduced PEX, a symbolic execution-based approach primarily targeting C#/.NET. Symbolic execution involves exploring possible code paths by treating input values as symbols. Conditional branches (e.g., if, while) generate constraints (e.g., $x > 0$), and a constraint solver is used to find concrete input values that satisfy or bypass these conditions. Symbolic execution is more systematic and deterministic than random testing, but it can suffer from scalability issues due to the path explosion problem, and it requires a constraint solver. To mitigate these challenges, symbolic execution is often combined with random testing or search-based techniques, in what is known as “concolic testing”.

2.2 AI-Driven Test Generation

With the rise of deep learning and natural language processing (NLP), AI-driven test generation methods have gained popularity. These approaches attempt to leverage machine learning models trained on large codebases to generate more intelligent and human-like test cases.

Tufano et al. [2] developed AthenaTest, an automated unit test generation tool that leverages the BART transformer model [3]. The tool was evaluated on five Defects4J projects, generating 25,000 passing test cases that covered 43.7% of focal methods in just 30 attempts. According to the authors, a developer survey revealed a strong preference for the readability, understandability, and effectiveness of tests generated by AthenaTest [2] compared to those produced by EvoSuite [1]. While AthenaTest generates test cases that are generally readable and easy to understand, the quality of the generated tests can vary. Some may fail to cover all relevant scenarios or miss certain defects.

Yuan et al. [16] investigated the ability of ChatGPT to automatically generate unit tests. While the generated tests demonstrated good coverage and readability comparable to human-written tests, they still exhibited issues related to accuracy, such as compilation errors and incorrect assertions. To address these limitations, the authors proposed ChatTester [16], a system that iteratively improves the initial test cases generated by ChatGPT. According to their results, ChatTester produces a higher number of compilable tests and more correct assertions compared to using ChatGPT alone.

Sapozhnikov et al. [17] introduced TestSpark, an open-source plugin for IntelliJ IDEA designed to facilitate unit test generation directly within the development environment. It leverages both traditional automated test generation techniques and large language models (LLMs) by establishing a feedback loop between the LLM and the IDE. Unlike tools such as EvoSuite [1] or AthenaTest [2], which are often complex or bound to a single technique,

TestSpark [17] emphasizes practical usability, allowing developers to easily modify, execute, and integrate the generated tests into their projects. Its extensible architecture also enables the addition of new generation strategies with minimal effort.

Chen et al. [11] introduced Codex, a GPT-based model specialized in code, and evaluated it on various code-related tasks such as function generation and bug fixing. They mention unit test generation as a use case, demonstrating that the model can produce plausible test code, though without any formal guarantees of correctness.

2.3 Positioning of Our Work

Despite advancements in search-based, random, and AI-driven test generation, a gap remains in the ability to generate readable, context-aware, and semantically correct test cases. We explore the Two-Step Zero-Shot Prompting approach with the aim of addressing these gaps, specifically by: (1) leveraging LLMs to generate test scenarios before generating test code to ensure more structured and contextualized test cases, (2) improving assertion accuracy by guiding the model to produce more meaningful and logically correct assertions, and (3) enhancing test readability and maintainability, to make generated tests more useful for developers. This work aims to bridge the gap between AI-driven test generation and traditional automated techniques by introducing a structured LLM-based methodology that combines test scenario identification with code synthesis for higher-quality test cases generation.

3 Two-step zero-shot prompting test generation

The current work introduces an approach that leverages the reasoning capabilities of Large Language Models (LLMs) to generate unit tests in a structured and contextualized manner.

Specifically, we used Mistral 7B model [18], built on the Transformer architecture [19]. With 7 billion parameters, Mistral 7B offers performances that surpass even larger models like the Llama 2 13B [20]. It was selected primarily for its ability to efficiently handle long prompts, thanks to its sliding window attention mechanism. This feature is particularly important in our context, where generating unit tests often involves incorporating method code, constructor signatures, and usage context within a single prompt. Additionally, although Mistral 7B is not specifically fine-tuned for code, it has shown competitive performance in reasoning and text generation tasks, including code synthesis. In our experiments, it produced more consistent and syntactically valid test cases than other open-source models of comparable size. Finally, being fully open-source makes Mistral 7B particularly attractive for research reproducibility and transparency, as it can be deployed locally and allows precise control over prompt engineering, model behavior, and evaluation unlike proprietary models such as GPT-4.

Zero-shot prompting [21] is a technique in which a language model is instructed to perform a task without having seen any specific training examples related to that task. The proposed approach breaks down the zero-shot prompting process into two distinct but complementary interaction steps with the model.

3.1 Motivations

Traditional test generation tools often operate as black-box systems, directly producing test code from source inputs without explicitly modeling the underlying logic or intent. In contrast, our approach mirrors the reasoning process of a human developer by first identifying what needs to be tested before deciding how to test it. This improves clarity, relevance, and assertion quality. Furthermore, zero-shot prompting [21] allows to bypass the need for training examples or fine-tuning, making the method highly adaptable.

3.2 Two-Step Zero-shot Prompting

Step 1 – Test Scenario Identification:

We first prompt Mistral 7B [18] to generate possible test scenarios for each specified method (refer to figure 1). Although the model does not reason in the strict sense, it is able to produce coherent scenarios by relying on patterns learned during training. In practice, this translates into an ability to "simulate" plausible uses of a method, identifies typical input/output cases, and suggests boundary conditions. The model is guided to analyze the target method and considers a variety of input conditions, execution paths, and output behaviors that may be relevant for comprehensive testing.

Step 2 – Test Code Generation:

After identifying the various test cases, the second step involves prompting the model to generate the corresponding unit tests for each identified test case. At this stage, the model is instructed to produce concrete test scripts that include the necessary assertions to validate the behavior of the method under test and any required setup code (see figure 1).

4 Experimental Evaluation

We present the descriptive statistics of the dataset used in our approach, along with comparative results obtained from EvoSuite.

4.1 Benchmark Selection

To evaluate our approach in a realistic and diverse context, we selected four projects from the SBST 2020 [22] unit test generation competition benchmark: GUAVA (https://github.com/google/guava) , SPOON (https://github.com/INRIA/spoon/), PDFBOX (https://github.com/apache/pdfbox), and FESCAR/Seata (https://github.com/apache/incubator-seata). These projects were chosen for their varied size, complexity, and domain coverage. GUAVA is a Java utility library developed by Google that provides advanced collections, caching mechanisms, and various tools for data manipulation. SPOON is a framework specifically designed for the analysis and transformation of Java code, relying on Abstract Syntax Trees (AST). In contrast, PDFBOX is a library dedicated to handling PDF files, offering functionalities for creating, modifying, and extracting content from PDFs. Finally, FESCAR is a distributed transaction management framework for microservices, enabling the coordination of multiple databases or systems.

This diversity allows to assess the generalizability of our method across different types of real-world software systems. Table 1 shows the descriptives statistics of the SBST 2020 projects.

Table 1 shows that SPOON has the highest total number of branches (1,072), despite having only 10 classes, which suggests a higher level of complexity per class relatively to remain systems.

```
### Instructions:
As a Java developer, in the context of unit testing, list a maximum of 5 relevant
scenarios to test for the specified method of the following class:
        <focal_method_and_context>
        Desired output format start by "Test scenarios":
        Test scenarios:
        1. Test for <scenario_1>
        2. Test for <scenario_2>
        ...
        x. Test for <scenario_x>
```

Figure 1: Test scenario identification

```
### System Message:
As an assistant, I need to help a Java developer write a unit test for all public methods
of the given class using JUnit 5.

### Requirements:
- Do not call private methods: Private methods should never be directly invoked in unit tests.
- Do not access private variables: Private variables in the class should not be accessed or manipulated directly in the
tests.
- Never Use mock.
- Follow the AAA pattern: Structure each test according to the Arrange (set up the test context), Act (execute the action
being tested), and Assert (verify the result) pattern.
- Cover critical use cases: Ensure that the tests cover key scenarios, including normal cases, edge cases, and error
scenarios.
- Import all necessary functionalities: Make sure to import all classes and functions that are required to perform the
tests.
- Never instantiate an abstract class.

### Instruction:
Generate unit test that covers all the following test scenarios for the specified method
of a given class using JUnit 5.

<generated_scenarios_first_step>

### Java Class with the specified method:

<focal_method_and_context>
```

Figure 2: Step 2 - Test code generation

Table 1: Descriptive statistics of the SBST 2020 Projects

Projects	#Classes	#Branches	Mean	SD
FESCAR	20	490	24.5	27.3
GUAVA	20	926	46.3	61.5
PDFBOX	20	1070	53.5	75.6
SPOON	10	1072	107.2	12.9

PDFBOX and GUAVA also exhibit a high number of branches, with an average of 46.3 and 53.5 branches per class, respectively. FESCAR, on the other hand, has the lowest total number of branches (490) and a very low median (11), which may indicate that branch complexity is located in a smaller subset of classes. It is worth noting that these 70 classes (20 from each of the GUAVA, FESCAR, and PDFBOX projects, and 10 from SPOON) were selected through a filtering process. Due to time and feasibility constraints, not all classes from the original projects were included. Instead, a selection procedure was applied to retain only a representative subset of classes with sufficient complexity and compatibility with test generation tools [22].

4.2 Evaluation Metrics

To evaluate the effectiveness of the generated unit tests, we selected and adapted JUGE framework [23], a robust infrastructure specifically designed to test and assess the performance of Java test generation tools. This platform enables accurate evaluation through its structural coverage and mutation analysis capabilities, thereby providing a reliable measure of the fault detection capabilities of the generated test suites. It has been used in several editions of competitions related to automated unit testing, co-organized with workshops such as the Search-Based Software Testing Workshop (SBST) [24]. JUGE [23] evaluates the quality of unit tests using several key metrics.

- Line coverage: This metric measures the percentage of executable lines of code covered by tests. A high line coverage indicates that the tests exercise a large portion of the code.
- Branch coverage: This metric computes the percentage of code branches reached by the test controle flow. A high branch coverage shows that the tests verify all possible logical conditions.
- Mutation score: Mutation analysis involves generating slightly modified versions of the code, known as mutants, to check whether the tests can detect these changes. These modifications simulate potential errors or faults, such as changing a logical operator, altering a constant, or modifying a condition. The main goal of this technique is to assess whether the existing tests can catch these changes. If a test fails to detect a mutant (i.e., the mutant survives), it indicates that the test may not be robust enough to catch similar errors in a real-world scenario. Mutation score indicates the percentage of artificially introduced faults (mutants) that are detected (killed) by the generated tests.

The three metrics reflect the fault-detection potential and the depth of exploration of the tested code.

4.3 Results and comparative analysis

Table 2 presents a comparison of these metrics scores between EvoSuite, a widely used genetic algorithm-based tool, and our approach, Two-Step Zero-Shot Prompting (2SZSP).

EvoSuite achieved significantly higher line and branch coverages, with differences of 12.7% and 18.2% respectively. This is consistent with its optimization objective, which is explicitly coverage driven.

However, our approach significantly outperforms EvoSuite in terms of mutation score (+10.5 percentage points). This suggests that while our tests may cover fewer lines or branches, they are more effective at detecting faults, which supports the value of LLM-guided scenario-based test generation.

Table 2: Results: Evosuite vs. Our Approch

Metrics	EvoSuite	2SZSP
Line Coverage (%)	56.1	43.4
Branch Coverage (%)	49.5	31.3
Mutation score (%)	30.8	41.3

5 Discussion

This section discusses the results obtained from our experiments, highlighting the strengths and limitations of the proposed approach considering the evaluation metrics.

5.1 Strengths of 2SZSP Approach

- Clarity and Structure of Test Code: One of the key strengths of our method lies in its ability to generate context-aware unit tests. Unlike traditional tools that focus primarily on structural coverage, the LLM-based approach considers the semantic context and interdependencies between methods and classes. This enables it to uncover subtle faults that might be missed by coverage-driven strategies. In particular, the higher mutation score achieved by our approach, 41.3%, compared to EvoSuite, 30.8%, indicates that the generated tests are not only syntactically correct but also more effective at exposing meaningful faults.

Although code coverage is widely used as an indicator of test quality, several studies have shown that it can be misleading when used in isolation. A test may execute a line of code without actually verifying its effective behavior. This phenomenon is sometimes referred to as a “vanity metric” [25], highlighting that coverage may give a false sense of security. In contrast, the mutation score provides a more accurate measure of a test suite’s ability to detect faults. This metric is based on the generation of mutants (small syntactic changes in the source code) and observes whether the existing tests can cause these mutants to fail. If a test passes despite a modified version of the code, it indicates a lack of effective assertions or weaknesses in the test design. Recent studies confirm that mutation score provides a more fine-grained measure of test effectiveness than code coverage alone. Jain et al. [26] introduced the notion of the “oracle gap”, which highlights discrepancies between coverage and mutation score — revealing parts of the code that are poorly tested despite being covered. Additionally, Pedro Rijo [27] and BellSoft [28] emphasize that mutation testing encourages developers to write truly meaningful assertions, ultimately making the test suites more robust and reliable.

- Semantic Effectiveness in Fault Detection: Another notable strength of our approach lies in the readability of the generated tests. It is important to emphasize that the tests produced by our method are not only more effective in detecting faults, but also easier to understand and interpret. Test readability is a critical factor for ensuring maintainability,

Test scenarios:

1. Test for adding a valid non-empty collection
2. Test for adding an empty collection
3. Test for adding a null collection
4. Test for adding a collection with duplicate éléments
5. Test for adding a collection that exceeds the queue's maximum size

Figure 3: Automatically generated test scenarios for the addAll(Collection) method

```
@Test
public void testAddAllNullCollection() {
    MinMaxPriorityQueue var1 = MinMaxPriorityQueue.create();
    Assertions.assertThrows(NullPointerException.class, () -> {
      var1.addAll((Collection)null);
    });
}
```

Figure 4: Generated unit test for scenario 3 (null input) of the addAll(Collection) method

```
@Test
public void testAddAllWithCollectionOfElementsAndModifiedFlag() {
    MinMaxPriorityQueue var1 = MinMaxPriorityQueue.create();
    List var2 = Arrays.asList(1, 2, 3);
    boolean var3 = var1.addAll(var2);
    Assertions.assertTrue(var3);
    Assertions.assertEquals(var2, var1);
}
```

Figure 5: Incorrectly generated test for scenario 1 (non-empty input) of the addAll(Collection) method

developer adoption, and long-term reliability of test suites in real-world software projects.

Several studies have shown that poorly readable tests hinder developers' ability to understand the test's intent, detect logical errors, and confidently evolve the system. For instance, Sedano, T. [29] highlights that improved readability helps programmers modify and correct code more effectively, while reducing the risk of introducing regressions.

In our context, this benefit stems from the scenario-driven structure of our two-step prompting method, which encourages the generation of semantically explicit and well-named test cases, closer to human-written tests. This characteristic enhances the trustworthiness and usability of the generated code for real development workflows.

Figures 3 and 4 illustrate the generated test scenarios and corresponding test cases for the *addAll(Collection)* method of the *MinMaxPriorityQueue* class (which belongs to the GUAVA project), specifically focusing on scenario 3: handling a null input collection. This method inserts all elements from a given collection into the priority queue while preserving its internal ordering constraints. Figure 3 shows a set of five well-defined test scenarios derived from the method specification. It demonstrates that the model can identify relevant and diverse conditions including edge cases such as empty or oversized collections which contributes to more robust testing strategies. Figure 4 sows a generated unit test that correctly handles the null input case (scenario 3) by asserting that a *NullPointerException* is thrown. The test is concise, readable, and follows common JUnit conventions, showcasing the ability of the model to produce meaningful and properly structured test cases from a simple prompt.

5.2 Approach's Limitations

Despite its strengths, the proposed Two-Step Zero-Shot Prompting approach also presents several limitations that should be acknowledged.

- Unusable Test Cases: One major limitation lies in the high proportion of non-compilable or invalid test cases. In our experiments, approximately 64% of the generated tests failed

to compile, indicating that they did not satisfy even the most basic syntactic requirements. This issue significantly reduces the immediate utility of the generated tests, as non-compilable code cannot be executed or validated. The problematic cases often involved: missing imports or undeclared variables, incomplete object instantiations, invalid method calls or incorrect argument types. This suggests that LLMs still lack robust control over language-level constraints, particularly when generating code in complex or unfamiliar contexts. Enforcing compilation policy in finetuning step could mitigate the structural error of generated test codes.

Figure 5 presents a generated test for the *addAll(Collection)* method of the *MinMaxPriorityQueue* class that targets scenario 1: adding a valid non-empty collection (see figure 3). While the test attempts to verify that the queue is modified as expected, it introduces a semantic error by comparing a *List* directly to a *MinMaxPriorityQueue* using *assertEquals*. This is logically incorrect due to type incompatibility and internal ordering differences.

- Prompt Engineering Complexity: Another challenge encountered during this study is related to the complexity of prompt engineering. While zero-shot prompting eliminates the need for labeled training data, or providing examples as in few-shot prompting, which can overload the prompt and reduce clarity, it still requires carefully crafted instructions to guide the LLM effectively. Small variations in wording, context structure, or instruction clarity can lead to significant differences in output quality.

5.3 Opportunities for improvement

While our approach shows promising results, there are several avenues for improvement that could enhance its reliability and scalability.

One key challenge lies in the hallucination phenomenon [30] inherent to Large Language Models. LLMs can sometimes generate syntactically valid but semantically incorrect code, including method calls that do not exist, miss use of APIs, or logically incoherent assertions.

This issue affects both the compilability and the trustworthiness of the generated tests. Future work could explore post-processing mechanisms, such as static code analysis or integration of adapted grammar compiler like JAVACC [31, 32], to automatically detect and filter out hallucinated elements patterns.

Another area of improvement involves automating error correction. A post-generation pipeline that includes syntactic validation and auto-fixes (e.g., adding missing imports, correcting argument types) could significantly reduce the number of unusable test cases.

Another promising direction would be to combine LLM-based generation with static analysis techniques. While LLMs excel at capturing context and intent, they may produce syntactically correct but semantically invalid code. Integrating static analyzers could help validate the generated tests, detect issues such as missing dependencies or unreachable code, and even provide feedback to refine prompts or filter outputs. Such a hybrid approach would bridge semantic reasoning and formal code correctness, improving the overall precision and reliability of the generated test suites.

6 conclusion

This paper introduced a structured Two-Step Zero-Shot Prompting approach for unit test generation using LLMs. Our experiments on SBST 2020 benchmarks [22] show that, while code coverage remains slightly lower than EvoSuite, our method achieves a higher mutation score and produces more readable and context-aware tests. These results highlight the potential of LLMs to improve fault detection and test maintainability. However, challenges remain, notably the rate of non-compilable tests and sensitivity to prompt design.

Future work will explore post-processing pipelines, static analysis integration, and hybrid approaches to improve robustness and automation.

References

[1] Fraser, G. and A. Arcuri. Evosuite: automatic test suite generation for object-oriented software. in Proceedings of the 19th ACM SIGSOFT symposium and the 13th European conference on Foundations of software engineering. 2011.

[2] Tufano, M., *et al.*, Unit test case generation with transformers and focal context. arXiv preprint arXiv:2009.05617, 2020.

[3] Lewis, M., Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. arXiv preprint arXiv:1910.13461, 2019.

[4] Shamshiri, S., *et al.* Do automatically generated unit tests find real faults? an empirical study of effectiveness and challenges (t). in 2015 30th IEEE/ACM International Conference on Automated Software Engineering (ASE). 2015. IEEE.

[5] Shamshiri, S., *et al.* How do automatically generated unit tests influence software maintenance? in 2018 IEEE 11th international conference on software testing, verification and validation (ICST). 2018. IEEE.

[6] Alagarsamy, S., C. Tantithamthavorn, and A. Aleti, A3test: Assertion-augmented automated test case generation. Information and Software Technology, 2024. **176**: p. 107565.

[7] Yang, L., *et al.* On the evaluation of large language models in unit test generation. in Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering. 2024.

[8] Barr, E.T., *et al.*, The oracle problem in software testing: A survey. IEEE transactions on software engineering, 2014. **41**(5): p. 507-525.

[9] Achiam, J., *et al.*, Gpt-4 technical report. arXiv preprint arXiv:2303.08774, 2023.

[10] Feng, Z., *et al.*, Codebert: A pre-trained model for programming and natural languages. arXiv preprint arXiv:2002.08155, 2020.

[11] Chen, M., *et al.*, Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374, 2021.

[12] Pacheco, C. and M.D. Ernst. Randoop: feedback-directed random testing for Java. in Companion to the 22nd ACM SIGPLAN conference on Object-oriented programming systems and applications companion. 2007.

[13] Panichella, A., F.M. Kifetew, and P. Tonella, Automated test case generation as a many-objective optimisation problem with dynamic selection of the targets. IEEE Transactions on Software Engineering, 2017. **44**(2): p. 122-158.

[14] Panichella, A., F.M. Kifetew, and P. Tonella. Reformulating branch coverage as a many-objective optimization problem. in 2015 IEEE 8th international conference on software testing, verification and validation (ICST). 2015. IEEE.

[15] Tillmann, N. and J. De Halleux. Pex–white box test generation for. net. in International conference on tests and proofs. 2008. Springer.

[16] Yuan, Z., *et al.*, Evaluating and improving chatgpt for unit test generation. Proceedings of the ACM on Software Engineering, 2024. **1**(FSE): p. 1703-1726.

[17] Sapozhnikov, A., *et al.* TestSpark: IntelliJ IDEA's Ultimate Test Generation Companion. in Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings. 2024.

[18] Jiang, A.Q., *et al.*, Mistral 7B. arXiv preprint arXiv:2310.06825, 2023.

[19] Vaswani, A., *et al.*, Attention is all you need. Advances in neural information processing systems, 2017. **30**.

[20] Touvron, H., *et al.*, Llama 2: Open foundation and fine-tuned chat models. arXiv preprint arXiv:2307.09288, 2023.

[21] DAIR.AI. 2025; Available from: https://www.promptingguide.ai/techniques/zeroshot.

[22] Devroey, X., S. Panichella, and A. Gambi. Java unit testing tool competition: Eighth round. in Proceedings of the IEEE/ACM 42nd International Conference on Software Engineering Workshops. 2020.

[23] Devroey, X., *et al.*, JUGE: An infrastructure for benchmarking Java unit test generators. Software Testing, Verification and Reliability, 2023. **33**(3): p. e1838.

[24] SBST 2020. 2020; Available from: https://sbst20.github.io/.

[25] Codecov, Mutation Testing: How to Ensure Code Coverage Isn't a Vanity Metric. 2022.
[26] Jain, K., *et al.* Mind the gap: The difference between coverage and mutation score can guide testing efforts. in 2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE). 2023. IEEE.
[27] Rijo, P., An intro to Mutation Testing - or why coverage sucks. 2019.
[28] Bellsoft, A Comprehensive Guide to Mutation Testing in Java. 2025.
[29] Sedano, T. Code readability testing, an empirical study. in 2016 IEEE 29th International conference on software engineering education and training (CSEET). 2016. IEEE.
[30] Xu, Z., S. Jain, and M. Kankanhalli, Hallucination is inevitable: An innate limitation of large language models. arXiv preprint arXiv:2401.11817, 2024.
[31] Community, J. JAVACC. Available from: https://javacc.github.io/javacc/.
[32] Revusky, J., Introducing the JavaCC 21 parser generator for Java. 2021.